

Travigenie : Telangana Tourism Chatbot

I. Setup Guide

1. Install System Dependencies

Linux

- `sudo apt update`
- `sudo apt install -y python3-pip python3-venv # (prefer Python 3.11.9)`

Windows

- Download Python from the official website: <https://www.python.org/downloads/>

WSL (Windows Subsystem for Linux)

- `cd /mnt/c/path/to/project`
- `sudo apt update`
- `sudo apt install -y python3-pip python3-venv # (prefer Python 3.11.9)`

2. Create and Activate a Virtual Environment

- `python3 -m venv .venv`
- **Linux:** `source .venv/bin/activate`
- **Windows:** `.venv/Scripts/activate`

3. Install Ollama and Pull Mistral Model

Linux/wsl

- `sudo apt install curl`
- `curl -fsSL https://ollama.com/install.sh | sh`
- `ollama serve`
- `ollama pull mistral:7b-instruct-q4_K_M` (v0 version is stable but not versatile)

Windows

- Install Ollama via the official guide: <https://ollama.com/download>
- Open Ollama and serve the model from the command line.

4. Install Python Packages

`pip install -r requirements.txt`

requirements.txt should contain:

streamlit, requests, fastapi, uvicorn, python-multipart, langchain, langchain-community, langchain-core, chromadb, sentence-transformers, unstructured, PyMuPDF, ollama, openpyxl, python-docx, litellm, crawl4ai, playwright, pydantic

5. How to Run the Files

Run only server

- run `chmod +x run_all.sh`
- And then `./run_all.sh`

Run the API Backend (Terminal - 1)

- `uvicorn app:app --reload`
- This starts the FastAPI server at <http://localhost:8000>.

Run the Streamlit GUI ([client.py](#)) (Terminal - 2)

- `streamlit run client.py`
- This opens the chatbot interface in your default browser.

Run the CLI Interface ([CLL.py](#)) (Terminal - 2)

- `python3 CLL.py`
- This allows text-based interaction via terminal.

Upload Documents for Vector Indexing

- `python3 data_tagging.py`
- Used to preprocess and store document embeddings in ChromaDB.

Run RAG Chat (Optional Component)

- `python3 rag_chat.py`
- Useful for direct testing of the Retrieval-Augmented pipeline.

II. Project Structure and Functional Overview

Architecture: Client-Server (API + GUI/CLI)

Core Files

1. [main.py](#)

- Central controller for the chatbot.
- Handles validation, web scraping, and data retrieval.
- Acts as API backend using FastAPI.

2. [client.py](#)

- **Streamlit GUI**
- Displays chatbot interface with user query input, results display, and error handling.
- Sends POST requests to the backend `/chat` endpoint.

3. `CLI.py`

- **Terminal Interface**
- Lightweight text-based interaction tool.
- Displays welcome message, processes user input, calls API and prints formatted results.

4. `app.py`

- **Main server-side code**
- Initializes FastAPI app with proper CORS and endpoints.

5. `rag_chat.py`

- Manages **Retrieval-Augmented Generation (RAG)** pipeline.
- Combines embeddings from local sources + scraped content.

6. `data_tagging.py`

- Processes uploaded documents.
- Chunks data, tags it with activity type, and stores it in ChromaDB.

III. Features and Modules

1. Document Upload and Vector Storage

- Upload PDF/Excel files.
- Chunk and tag documents into categories:
 - Family activities
 - Adventure
 - Historic tours
- Stored as embeddings in **ChromaDB** using Ollama + sentence-transformers.

2. AI Chat System

- Uses **Ollama Mistral 7B** (Quantized version) for response generation.
- Integrates both RAG and LLM outputs.
- Performs validation using:
 - JSON-based keyword validator
 - LLM-based semantic check (fallback)

3. Query Validators

Travigenie ensures that only relevant and appropriate queries are processed using a **hybrid validation approach**, combining fast rule-based checks with semantic LLM understanding.

JSON-Based Fast Validator (Stage 1)

A lightweight, rule-based check is performed using a local JSON configuration file:

```
{
  "min_query_length": 5,
  "block_keywords": ["spam", "hack", "illegal", "adult", "explicit"],
  "telangana_keywords": [
    "telangana", "hyderabad", "warangal", "nizamabad", "khammam",
    "charminar", "golconda", "ramoji", "hussain sagar", "birla mandir",
    "salar jung", "bhadrachalam"
  ]
}
```

Logic:

1. If the query length is **less than 5 characters**, it's marked as too short and rejected immediately.
2. If the query contains any **blocked keyword**, it is flagged as inappropriate and rejected.
3. If at least one **Telangana-related keyword** is found, the query is considered **valid**.

LLM-Based Semantic Validator (Stage 2)

If the JSON check fails to confidently validate, Travigenie invokes the **Mistral 7B model via Ollama** to semantically analyze the query using a custom prompt like:

“Check whether the following query is about Telangana tourism, travel, or history. Return ‘VALID’ or ‘INVALID’.”

Advantages of this step:

- Handles queries with indirect or implicit references to Telangana (e.g., “Tell me about the state known for Charminar”).
- Adds robustness by catching relevant queries missed by keyword-only methods.
- Helps in generalization and supports natural language understanding.

4. Web Scraping System (Crawl4AI)

Goal:

Scrape real-time content from trusted tourism sources when the local knowledge base is insufficient.

Target Sites:

- <https://www.incredibleindia.org/>
- <https://www.telanganatourism.gov.in/>
- <https://www.tripadvisor.in/>

Extraction Strategy:

- Async Web Crawler
- 45-sec timeout
- Retry logic (up to 3 times)
- Extracts:

```
{  
  "source": "Website name",  
  "data_summary": "Travel info",  
  "url": "source-url",  
  "extracted_image_urls": ["img1.jpg"]  
}
```

Trigger Conditions:

- Vector similarity score < 0.6
- Keywords: "current", "open", "weather", "deals", "events"

5. External API Integrations

- **Pixabay API:** Travel image fetching
- **Amadeus API:** Hotel booking via city code lookup
- Proper error, timeout, and fallback handling included

6. Smart Query Understanding

- Detects context like:
 - "family-friendly spots"
 - "adventure tours"
 - "historic places near Warangal"
- Automatically chooses between static and scraped data

7. Response Formatting

- Adds headers, formatting, and location highlights
- Merges content from scraped + static sources
- Displays images when available

IV. Improvements Made

1. Response Quality

- Switched from GUI to CLI to reduce latency
- Shifted from basic models to **quantized Mistral** for fast inference
- Tried DeepSeek but found compatibility issues

2. Performance Boosts

1. Parallelisation Methods Used:

- Manual multiprocessing using threads
- Python-native: `joblib`, `aiohttp`, `asyncio`
- Numba with OpenMP

Method	What It Does	Speed	Difficulty	Setup Needed	Best For	Uses	
Serial	Runs one task at a time	Slowest	Very Easy	None	Small scripts, debugging, testing	CPU	
Manual Parallel	Uses <code>threading</code> or <code>multiprocessing</code> manually	Medium	Medium	No extra install	Custom parallel tasks, I/O or CPU-based jobs	CPU	
Python-native Parallelism	Uses <code>joblib</code> , <code>aiohttp</code> , <code>numpy</code> for easy parallel tasks	Fast	Easy	<code>pip install joblib aiohttp numpy</code>	Embeddings, HTTP calls, vectorized math	CPU	
Numba with OpenMP	Speeds up loops using Numba and runs them on multiple cores	Very Fast	Medium	<code>pip install numba</code>	Numerical computations, array operations	Multi-core CPU	
Cython with OpenMP	Compiles Python to C and uses OpenMP for parallel execution	Very Fast	Hard	Cython setup, compiler, OpenMP flags	Performance-critical loops and inner kernels	Multi-core CPU	
CUDA with GPU	Runs code on the GPU using CUDA via Numba or PyTorch	Extremely Fast	Hard	CUDA Toolkit, GPU drivers	Heavy computations, ML tasks, matrix ops	NVIDIA GPU	

3. Enhanced Data Quality

- Uploaded bulk PDFs/Excels on Telangana tourism
- **Tagging:**
 - Labeled documents by content (family/adventure)
- **Cleaning:**
 - Removed noise, repeated headers, and advertisements
 - Unified headings, location names
- **Embedding Update:**
 - Re-generated embeddings after data refinement

V. Endpoints Summary

Endpoint	Method	Purpose
<code>/upload-docs</code>	POST	Upload tourism documents
<code>/chat</code>	POST	Main chatbot interaction
<code>/get-image</code>	POST	Image retrieval based on query
<code>/book-hotel</code>	POST	Fetch hotels via Amadeus API

VI. Future Work

- Add session memory to CLI & Streamlit
- Auto-summarize long documents
- Add Telugu/Hindi language support
- Full image gallery + map view in GUI
- React Integration

VII. Conclusion

Travigenie is a context-aware, real-time travel chatbot tailored for Telangana. With its hybrid validation, smart scraping, and fast model architecture, it provides both historical depth and current accuracy. Designed for flexibility, performance, and expansion.